

Daily Scholar Papers Report — 2026-07-14

2026-07-14

Daily Scholar Papers Report — 2026-07-14

[Download PDF](#)

Window covered: 2026-07-13 → 2026-07-14 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Four Scholar-alert threads landed in the window — one each from followed researchers Zhiqiang Lin, Cheng Wen, and Andreas Zeller, plus one Recommended-articles batch of three — for six raw candidates total. The user-curated forward queue was empty. Five candidates cleared Stage-1: two SSBSE 2026 short papers on search-based testing (the FANDANGO evolutionary grammar-fuzzer and an AFL-based fuzzer targeting the encoded AndroidManifest.xml), a SETTA 2026 LNCS chapter on retrieval-augmented LLM proof synthesis for Rust/Verus, an arXiv preprint accepted to ITC 2026 that recasts CPU fuzzing as hierarchical reinforcement learning over program layouts, and an ICML-adjacent position paper arguing for typed “web verbs” over DOM-level clicks in web agents. One recommended candidate was dropped at Stage-1 as a saturated LLM-for-vulnerability-detection empirical comparison. Preference layer remains quiet — `venue::preprint` (posterior 0.75, n=2) is the only attribute with enough observations to weigh in, and it only mildly reinforces the two arXiv candidates that were already followed-researcher / venue-strong Proceeds; no candidate crossed the 0.30 auto-Skip threshold.

Outstanding: 2 · **Keep:** 2 · **Borderline High-Priority:** 1

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	Search-Based Generation of Complex Inputs with FANDANGO	J. A. Z. Amaya, M. Smytzek, A. Zeller	SSBSE 2026 (LNCS 16699), pp. 151–154	DOI
1.2	HiFuzz: Hierarchical Reinforcement Learning for Semantic-Aware and Adaptive CPU Fuzzing	Y. Wang, H. Fan, Z. Liu, X. Zhou, Y. Lyu, J. Xu, W. Zhang	arXiv 2607.06619 (cs.AR); accepted, ITC 2026	arXiv
2.1	Enhancing LLM-Based Proof		SETTA 2026, Shanghai, July 4–6,	Springer

#	Title	Authors	Venue	Link
	Synthesis for Rust Programs via Semantic Chunking and Hierarchical Context	C. Wen, Z. Xu, D. Liu, J. Cao, Y. Liu, S. Qin, C. Tian	2026 (Springer LNCS)	
2.2	ApkFuzz: Search-Based Fuzzing for Android APK Vulnerability Discovery	K. Even-Mendoza, A. Dakhama, H. Berger	SSBSE 2026 (LNCS 16699), pp. 135–141	DOI
3.1	Position: Web Agents Should Use Typed Actions Instead of Click-Based Browsing	L. Jiang, R. Xi, Z. Liu, S. Chen, Z. Lin, S. Nath	arXiv 2602.17245 (cs.AI), 2026	arXiv

1. Outstanding

1.1 · SEARCH-BASED FUZZING · Grammar + Python constraints + evolutionary search delivers up to three orders of magnitude speedup over symbolic input generation at 100% input validity and equal-or-higher grammar coverage — with 100k+ PyPI installs and Volkswagen/Bosch industry adoption

1.1 [Search-Based Generation of Complex Inputs with FANDANGO](#) — J. A. Z. Amaya, M. Smytzek, A. Zeller (CISPA) — SSBSE 2026, LNCS 16699, pp. 151–154

Venue. International Symposium on Search-Based Software Engineering 2026 (SSBSE 2026), Springer LNCS 16699. Short tool paper (4 pages).

Problem. Generating semantically valid inputs to fuzz format-heavy targets (protocols, file formats, DSLs) is the long-standing weak spot of coverage-guided fuzzing: byte-level mutation rarely survives a parser, and hand-tuned generators do not compose. Symbolic techniques such as ISLa give strong validity guarantees but at execution costs that limit their throughput on realistic workloads.

Method — grammar × constraints × evolution. FANDANGO frames input generation as a search problem in the language of a context-free grammar decorated with **fully specified Python constraints**. Constraints are ordinary Python predicates over parse-tree fragments — so authors can freely mix data invariants (checksums, length fields, timestamps), semantic invariants (a referenced ID exists, a state is reachable), and target-specific oracles. The search itself is an evolutionary loop over parse trees: candidates that partially satisfy the constraint set are mutated and recombined via grammar-aware operators; fitness is a graded distance to full constraint satisfaction.

Headline numbers (from abstract). Versus the symbolic state of the art, FANDANGO reports **speedups of up to three orders of magnitude** while maintaining **100% input validity** and **equal or higher grammar coverage**. Adoption signals: **>100,000 PyPI downloads**, industrial users including Volkswagen and Bosch, and coverage in the science press.

Roadmap the paper flags. Two active extensions are called out explicitly: whitebox fuzzing (coverage-guided FANDANGO) and stateful fuzzing via *interaction grammars* that model multi-turn protocol behaviour — a natural bridge to the network-fuzzing and API-fuzzing lines. Related citations point back to Steinhöfel & Zeller’s “input invariants” (FSE 2022) and forward to Liggesmeyer/Amaya/Zeller “language-based protocol testing” (arXiv:2509.20308) — that arXiv preprint is the technical companion to read next.

Formal characterisation (professional prose). A FANDANGO specification is a pair (G, C) of a context-free grammar G and a set of Python constraint predicates C . The search seeks parse trees $t \in L(G)$ maximising $|\{c \in C : c(t)\}|$; when the maximum equals $|C|$ the tree is a semantically valid test input. Evolutionary variation operators are grammar-typed so mutation/crossover preserve well-formedness with respect to G . (Paraphrase — the tool paper does not display a labelled formula.)

Reusability.

- **Grammar-typed evolutionary operators over parse-tree fragments** are directly reusable in any grammar-fuzzer that today mutates strings or bytes — a drop-in improvement in validity for smart-contract IDL, protocol-buffer schemas, PDF, and OS-image formats.
- **Python-native constraint DSL** avoids the closed constraint language of ISLa/JSON-Schema fuzzers; teams can express oracle conditions in the same code they already use to write parsers and unit tests.
- **Distance-based fitness on partial constraint satisfaction** is transportable to symbolic and hybrid pipelines: emit ISLa/Z3 constraints as Python callables and let evolutionary search replace SMT-based enumeration when SMT stalls.

Cross-paper connection. FANDANGO and paper 2.2 (ApkFuzz) both land in the same SSBSE 2026 volume and together triangulate two ends of search-based fuzzing: FANDANGO at the *high-structure* end (grammar-typed evolution over full parse trees) and ApkFuzz at the *encoded-binary* end (AFL bit-level mutation of a packed XML region). The methodological contrast — grammar-typed operators vs. bit-flip primitives — is a clean case study in “how much structure is enough.”

Closing citable line: “FANDANGO has established itself as a practical input generator.”

1.2 · CPU FUZZING · Recasts processor fuzzing as hierarchical RL — a Program Agent lays out global structure while a Basic Block Agent fills instructions — with an adaptive coverage reward and a semantic-aware BB encoder that supplies intrinsic reward, outperforming state-of-the-art fuzzers on three real RISC-V cores; accepted at ITC 2026

1.2 HiFuzz: Hierarchical Reinforcement Learning for Semantic-Aware and Adaptive CPU Fuzzing — Y. Wang, H. Fan, Z. Liu, X. Zhou, Y. Lyu, J. Xu, W. Zhang (HKUST + HKUST-GZ) — arXiv 2607.06619 (cs.AR), 2026; accepted, IEEE International Test Conference (ITC) 2026

Venue. arXiv preprint, cs.AR (primary); accepted at IEEE International Test Conference 2026. Preference-neutral (venue::preprint posterior 0.75, n=2 — mild positive tilt, not decisive).

Problem. Coverage-guided fuzzers for processors (RFuzz, DifuzzRTL, ProcessorFuzz, TheHuzz) operate by mutating an existing pool of assembly programs. On modern out-of-order and superscalar cores the depth of architectural state that matters (issue-queue occupancy, ROB reordering, MSHR pressure, cache-coherence transitions) is not reachable by local mutation of an opaque seed program — the

mutant either fails to assemble, decays to already-covered layouts, or fails to exercise the interaction of interest. Reward for RL-based fuzzers is also sparse: coverage counters flip rarely and offer little gradient to a policy that must choose from a large action space.

Method — two-layer HRL for structured program generation. HiFuzz replaces mutation with *generation* under a hierarchy of two RL policies:

- **Program Agent** — the top-level policy — chooses the *global layout* of the program under test: number and types of basic blocks (arithmetic-heavy, memory-heavy, branch-heavy), control-flow topology, and coarse register-file usage. Its actions determine which architectural corners are stressed.
- **Basic Block Agent** — the low-level policy — receives the layout and *fills each block* with concrete RISC-V instructions consistent with the block’s declared role and with previously emitted context (register liveness, memory footprint, branch targets). This decomposition matches the natural block/CFG structure of a program and dramatically shrinks the effective per-step action space.

Reward shaping. Two complementary reward signals address the sparsity problem:

- **Adaptive coverage reward** — a coverage-derived extrinsic signal that reweights past-hit counters to keep pressure on genuinely under-explored regions rather than repeatedly rewarding easy re-hits.
- **Semantic-aware basic block encoder** — a learned encoder that embeds each basic block into a semantic space; the intrinsic reward is a novelty term derived from that embedding, so the Basic Block Agent gets useful feedback even when coverage counters are silent.

Evaluation. Three real-world RISC-V cores (the abstract phrases it as “extensive evaluations on three real-world RISC-V cores”). Metrics: coverage over time and bug detection versus state-of-the-art baselines. The paper reports HiFuzz “significantly outperforms state-of-the-art fuzzers in coverage and bug detection.” Specific numeric wins are not in the abstract; the full paper (accepted at ITC 2026) is the source of record.

Formal characterisation (professional prose). The MDP factorises: the outer policy $\pi_{hi}(a_{hi} | s)$ selects a layout descriptor a_{hi} (block types, count, topology); the inner policy $\pi_{lo}(a_{lo} | s, a_{hi})$ fills each block with instructions given the outer choice; total reward is a weighted sum of the adaptive coverage-derived extrinsic term and the encoder-derived intrinsic novelty term. (Paraphrase; the arXiv text is the authoritative statement.)

Reusability.

- **Hierarchical decomposition (layout policy over microstructure policy)** is directly portable to smart-contract fuzzers, kernel syscall fuzzers, and JIT fuzzers: any target where the *shape* of the test case (a call sequence, a syscall trace, an IR-op topology) is a separate design decision from the *filling*.
- **Semantic-aware block encoder as intrinsic reward** generalises to any fuzzer that emits structured units and wants a novelty signal that survives coverage plateaus — a compact answer to reward sparsity that does not require synthetic bug injection.
- **Adaptive coverage reweighting** is a small change to existing coverage-guided loops (AFL family, Fuzzilus family) that would benefit any target where a handful of easy-to-hit edges dominate the reward.

Cross-paper connection. With FANDANGO (1.1) and ApkFuzz (2.2), HiFuzz completes today’s search-vs-generation triple: FANDANGO evolves parse trees within a grammar; ApkFuzz bit-flips a packed binary region; HiFuzz generates whole programs from scratch under a learned hierarchical policy. The three read together as a spectrum of how much *a priori structure* the fuzzer commits to, and how the reward/fitness signal is engineered to work at that level.

Closing citable line: “we propose HiFuzz, a novel hierarchical reinforcement learning framework that replaces mutation with a [structured, two-layer generation process].”

2. Keep

2.1 · LLM PROOF SYNTHESIS · Attacks the context-window failure of AutoVerus-style Verus proof agents with (i) semantic-preserving chunking of Verus programs and (ii) hierarchical context assembly, so the LLM sees only the fragments and specifications relevant to the current proof obligation

2.1 [Enhancing LLM-Based Proof Synthesis for Rust Programs via Semantic Chunking and Hierarchical Context](#) — C. Wen, Z. Xu, D. Liu, J. Cao, Y. Liu, S. Qin, C. Tian — SETTA 2026 (Shanghai, July 4–6, 2026), Springer LNCS

Venue. Symposium on Dependable Software Engineering (SETTA) 2026, Shanghai. Springer LNCS chapter. Followed-researcher provenance (Cheng Wen).

Problem. Verus provides rigorous correctness guarantees for Rust programs, but writing the auxiliary specifications, invariants, and proof scripts by hand is expensive. LLM-based Verus agents such as **AutoVerus** demonstrate that proof synthesis can be automated in principle, but degrade sharply as programs grow: the relevant context (spec fragments, ambient predicates, prior proof steps) exceeds the LLM’s window and the agent either omits necessary facts or drowns in irrelevant ones. The paper’s thesis is that context *quality*, not context *size*, is the bottleneck — and that Verus programs have exploitable structure that a purpose-built assembler can use.

Method (as advertised in the abstract).

- **Semantic chunking.** The Verus program is split into chunks along **semantic** rather than syntactic boundaries — proof obligations grouped with the specs, predicates, and helper lemmas that participate in their discharge, rather than fixed-token windows or file-order chunks.
- **Hierarchical context assembly.** For each proof step the agent presents the LLM with a nested context: the immediate obligation and its local dependencies at the innermost level, then broader specification chapters, then global invariants — so the model sees a focused view first and can escalate scope only when local reasoning is insufficient.

Position in the literature. The AutoVerus baseline the paper builds on is the mainstream LLM-Verus agent line; the chunking-plus-hierarchy angle is complementary to the retrieval-augmented (KVerus) and self-evolution (Automated Proof Generation via Self-Evolution) branches that dominate the 2025–2026 arXiv line on this topic. Read it in that cluster.

Formal characterisation (professional prose). Given a Verus program P and an open obligation o , the pipeline computes a semantic-relevance function $sem(o, \cdot)$ over syntactic units of P and returns a chunk $C \subseteq P$ together with a partial order of context layers $L_1 \subset L_2 \subset \dots \subset L[\text{?}] = P$ used to escalate scope only

when the LLM's proof attempt at level L_i fails. (Paraphrase — the paper's precise definitions are the source of record.)

Reusability.

- **Semantic chunking over syntactic chunking** is directly applicable to any LLM-in-the-loop verification agent (Dafny, F*, Coq, Isabelle) — the payoff is proportional to how weakly a random-window baseline correlates with proof relevance.
- **Hierarchical context escalation** is a small change to any RAG pipeline: retrieve local first, fall back to global only on failure, with the retry driven by the agent's own diagnosis of missing facts.
- **Verus-specific chunk primitives** (spec / predicate / proof / impl blocks with their reference edges) are a portable schema for indexing verified codebases.

Cross-paper connection. Pairs naturally with LeanGuard (2026-07-12, 1.1) — both re-partition the LLM's role in a verification pipeline: LeanGuard by splitting *interpret* from *decide*, this paper by splitting *proof context* from *proof step*.

Closing citable line: “LLM agents such as AutoVerus show [promise] ...” (the abstract's opening frames the problem in exactly these terms; the paper's contribution is the semantic-chunking / hierarchical-context solution.)

2.2 · MOBILE FUZZING · Small, sharp AFL-based fuzzer with bit-level mutations on the encoded AndroidManifest.xml region of an APK finds 9 crashes / DoS conditions in production tooling — Google's apksigner and the widely used research tool DroidBot

2.2 [ApkFuzz: Search-Based Fuzzing for Android APK Vulnerability Discovery](#) — K. Even-Mendoza (KCL), A. Dakhama (Edinburgh), H. Berger (Ariel) — SSBSE 2026, LNCS 16699, pp. 135–141

Venue. SSBSE 2026, Springer LNCS 16699. Short paper (7 pages). Artifact on Zenodo (DOI 10.5281/zenodo.19476523).

Problem. APK-level security research is dominated by dex/native fuzzing; the **AndroidManifest.xml** file — which declares permissions, components, and metadata and gates every Android runtime decision — is a *high-trust, low-scrutiny* attack surface. Because it is encoded (binary XML) inside the APK, byte-level mutation of the APK archive can meaningfully corrupt manifest structure while leaving surrounding files intact, but generic APK fuzzers rarely target it.

Method. ApkFuzz is an AFL-based fuzzer that performs **bit-level mutations targeting the encoded region of AndroidManifest.xml**. The pipeline seeds AFL with valid APKs, treats the offset range corresponding to the encoded manifest as the sole mutation region, and lets AFL's coverage feedback (over the tool under test, not the app under test) drive exploration. The fuzzer is deliberately small — the value comes from putting AFL's mutation and scheduling primitives on the right slice of bytes.

Findings. Nine issues that cause crashes and enable **Denial-of-Service (DoS)** attacks in core Android tooling — including **Google's apksigner** and the widely used research tool **DroidBot**. The concrete finds against production and research tools (rather than yet another benchmark suite result) are what earns the Keep verdict.

Reusability.

- **Region-restricted AFL** — pin the fuzzer to a narrow byte range identified as high-trust / low-scrutiny — is a template that transports directly to iOS Info.plist, ELF metadata sections, PDF cross-reference tables, and Docker image manifests.
- **Small fuzzers against parsers of trusted metadata** are a productive research pattern precisely because the surrounding ecosystem has assumed the region is well-formed — the DoS finds against apksigner and DroidBot are evidence.
- The **artifact on Zenodo** ([10.5281/zenodo.19476523](https://zenodo.org/record/19476523/files/10.5281/zenodo.19476523)) lets independent teams reproduce the pipeline and rebase it onto adjacent metadata regions.

Cross-paper connection. See the 1.1 pairing — ApkFuzz and FANDANGO are the two ends of today's search-based-fuzzing spectrum in the same SSBSE 2026 volume.

Closing citable line: “We have identified 9 issues that cause crashes and enable Denial-of-Service (DoS) attacks in core tools.”

3. Borderline High-Priority

3.1 · WEB AGENT ARCHITECTURE · Position paper: replace DOM-level click/keystroke primitives with typed “web verbs” — functions with structured I/O, pre/postconditions, policy tags, and logging hooks — so agents synthesise short auditable programs instead of long brittle click chains

3.1 [Position: Web Agents Should Use Typed Actions Instead of Click-Based Browsing](#)
— L. Jiang, R. Xi, Z. Liu, S. Chen, Z. Lin, S. Nath — [arXiv 2602.17245 \(cs.AI\)](#), 2026

Venue. arXiv preprint. Followed-researcher provenance (Zhiqiang Lin, Microsoft Research adjacent). Preference-neutral — venue::preprint posterior 0.75, n=2 provides a mild positive tilt.

Problem. Today's web agents primarily operate through clicks, keystrokes, and DOM manipulation. That interface is *low-level* — long-horizon plans decay to brittle click chains that break when the DOM changes; every action costs a browser round-trip; and the resulting execution traces are opaque and hard to audit.

Position. Building a reliable agentic Web requires shifting from low-level interaction primitives to **typed actions** supported by a semantic layer. The authors propose **web verbs** as a concrete design: each verb exposes a web operation as a *typed function* with structured inputs, structured outputs, and documented behaviour. A verb can be backed by a server-side Web API or by a maintained client-side workflow — the distinction is transparent to the agent. Verb calls carry preconditions, postconditions, policy tags, and logging hooks; agents synthesise concise programs with explicit control flow and data flow, producing checkable execution traces.

Why it's on today's list. This is out of core scope (fuzzing / PL / vulnerability discovery), but the followed-researcher rule keeps it in and the framing is worth a page: the “web verbs” abstraction is close in spirit to WebAssembly-style capability APIs and to typed-effect systems in PL, and the pre/postcondition + policy-tag structure is exactly what a policy-checker or fuzzer would want to consume. Read as scaffolding for future *agent-security* work: adversarial reachability analysis over verb graphs, precondition-violation fuzzing, and policy-tag consistency checking are all downstream research questions the position paper opens.

Reusability.

- **Typed verb signatures with structured I/O** are a natural target for property-based testing and for formal reasoning about agent traces — much more so than DOM click logs.
- **Policy tags on verb calls** are a hook for information-flow analysis over web-agent workflows: taint sources and sinks become explicit.
- **Log hooks per verb** give a substrate for post-hoc auditing and offline replay — a concrete artefact that agent-security research can attack or defend.

Closing citable line (paraphrase from search-surfaced abstract): “A verb exposes a web operation as a typed function with structured inputs, structured outputs, and documented behavior.”

Cross-Paper Synthesis

Today’s five Proceed papers cluster into two coherent themes.

Search-based / RL-based fuzzing on a structure axis (1.1, 1.2, 2.2). FANDANGO (1.1), HiFuzz (1.2), and ApkFuzz (2.2) form a natural spectrum ordered by *how much a-priori structure the fuzzer commits to* before it starts searching. ApkFuzz is at the low-structure end: bit-flip mutations under AFL, restricted to a narrow byte region (the encoded AndroidManifest.xml). FANDANGO sits in the middle: parse trees typed by a context-free grammar are the search space, and grammar-typed evolutionary operators guarantee well-formedness. HiFuzz is at the high-structure end: whole programs are *generated* from scratch under a two-level RL policy that explicitly decides program layout before filling instructions. Each level of structure buys the fuzzer a different lever against the sparsity problem — ApkFuzz relies on AFL’s coverage over the tool under test; FANDANGO relies on graded constraint-satisfaction distance; HiFuzz adds a learned semantic-BB encoder for intrinsic reward. Read together, the three papers argue that the *right level* of structure is target-specific, but the *shape* of the argument — pick a structural unit, define grammar/policy/mutation over it, engineer reward against the sparsity it introduces — is transportable.

LLM-in-the-loop verification as a role-separation problem (2.1, and echoing 2026-07-12’s LeanGuard). Paper 2.1 (Cheng Wen et al.) and last week’s LeanGuard (2026-07-12, 1.1) are both attempts to fix LLM-driven verification by *decomposing the LLM’s role*. LeanGuard separates *interpret* from *decide* — the LLM filters AST facts but Lean 4 issues the verdict. Cheng Wen et al. separate *proof context* from *proof step* — a semantic chunker chooses what the LLM sees before it writes each proof line. Both papers agree that the LLM is not to be trusted with the whole job end-to-end, and that the productive engineering move is to isolate a well-defined subtask (fact selection; context assembly) where the LLM’s strengths are useful and its weaknesses are contained. The 2026 arXiv line (KVerus, VeruSyn, VCoT-Bench) is picking up the same message from other angles.

Bridge to 3.1. Paper 3.1’s typed “web verbs” abstraction is orthogonal on the surface, but shares the role-separation logic — the position paper argues the DOM-level primitives are too weak to reason about, and that any subsequent *security or verification analysis* of web-agent behaviour requires a typed interface first. It is the same argument as 2.1/LeanGuard translated from PL to web-agent engineering: pick the right interface to reason at, then let the analytic layer do its job.

Writing & Rationale Insights

“Position” papers can still earn their page. Paper 3.1 is a position paper with no experiments, and normally that would relegate it to a one-line summary. Two things push it into full-card treatment: (i) the followed-researcher rule (Zhiqiang Lin) — the point of the rule is precisely to catch signal from researchers whose framing tends to become future benchmarks; (ii) the *typed verb* abstraction is downstream-actionable — precondition/postcondition/policy-tag structure lets future research on agent security cite the position paper as scaffolding rather than as a target. Position papers are worth the space when they define a substrate future empirical work will use.

Reusability blocks over “impressive numbers” blocks. Three of today’s five papers are short (4–7 pages) and one is preprint-abstract-only. Depth from abstracts is thin — but the reusability blocks are not. For each paper the concrete question “what would a nearby team pull off this and reuse?” produces a longer and more useful list than “what did this paper report on its own benchmark?” Where the abstract gives only speedup ratios (FANDANGO’s 3-order-of-magnitude claim) or a count (ApkFuzz’s 9 crashes), the reusability block is what makes the summary transportable to a reader who does not care about the specific benchmark.

Cross-paper synthesis rewards contrast, not just overlap. The 1.1 / 1.2 / 2.2 spectrum works because the three papers *disagree* about the right level of structure — a synthesis that only listed “all three are fuzzing papers” would carry no information. The most durable cross-paper reads are the ones where the papers agree on the *shape* of the problem and disagree on the *knob to turn*; that’s what makes the synthesis useful when a future reader is choosing between them.